Minimum Revalidation Across Uncertain Dependency Histories

T

Abstract

Restoring an agent after a compromise requires checking which artifacts may have inherited suspect state. When incomplete logs admit several incompatible dependency histories, their edge union can invent paths and inflate revalidation cost without bound. We characterize when the union preserves the feasible recovery plans, establish the complexity of the scenario-wise problem, and give an exact witness-driven solver. An independent checker can verify optimality using path branches and rational prices. An alternative encoding shares repeated rebuild sets. The tests distinguish the size of a proof from the size of its encoding.

Summary and scope

Question. When can a recovery tool replace several plausible dependency histories with their edge union?

Finding. The replacement is exact if every union path contains the eligible vertex set of a path from a listed scenario. Otherwise the union can overstate minimum recovery cost by an arbitrarily large factor. A witness search and a separate certificate checker handle the scenario-wise problem directly.

Evidence and scope. The equivalence and certificate claims are proved, with finite exact checks against independent reachability and optimization oracles. The checker verifies the admitted histories. Whether those histories cover a real incident requires provenance evidence outside the optimization.

1 Revalidation as a recovery action

After a compromised tool adapter has been isolated, an operator needs to decide which artifacts to reconstruct before resuming service. Examples include cached tool outputs, serialized plans, and derived configuration. A reconstruction may be expensive even when the original artifact is small because it must start from authorized, independently validated inputs.

Let $\mathcal{G}$ be a finite, explicit family of plausible directed dependency graphs on a common vertex set. An edge represents possible dependence within the history it belongs to. Let s denote the origin of suspect state and t denote a target for resumption. Multiple origins or targets can be represented by a super-source or super-sink. Eligible vertices form a set U disjoint from $\{s, t\}$. Choosing $v \in U$ at cost $c(v) > 0$ replaces its state with a version independently revalidated against the authorized specification. The replacement discards suspect incoming dependencies and preserves downstream access to the new version. As far as transmission of existing taint is concerned, this operation removes v from every s-to-t path. Every true dependency history must belong to $\mathcal{G}$ for a guarantee about the actual recovery to follow.

Robust cut problems use multiple graph scenarios in several ways. Kakimura and Schlotter [3] study proximity to scenario-specific optima. Here the same set of reconstructions must interrupt all suspect paths in every scenario.

2 When the union is an exact recovery model

For a path P, let $I(P) = V(P) \cap U$. A revalidation set R works in a graph if and only if it intersects $I(P)$ for every s-to-t path in that graph. Equivalently, the target is unreachable from the suspect source after removing R. Paths with no eligible internal vertex certify that revalidation of these artifacts alone cannot authorize resumption.

Theorem 1. *A cut in the edge union of $\mathcal{G}$ always works in every scenario. The converse holds for every $R \subseteq U$ exactly when, for each union path Q, some path P in a single scenario satisfies $I(P) \subseteq I(Q)$.*

Proof. All scenario paths occur in the union, which gives the first implication. Under the stated condition a scenario-wise cut intersects $I(P)$ and therefore intersects $I(Q)$. Conversely, if a union path Q has no such scenario path, take $R = U \setminus I(Q)$. Each scenario path has an eligible vertex outside $I(Q)$, so R hits it. The set R misses Q. □

Thus the union can be exact even if some paths never occur in a single scenario, provided every such path contains a sufficient scenario constraint. Only inclusion-minimal eligible path sets matter to this test. If the condition fails and $I(Q)$ is nonempty, giving vertices in $I(Q)$ cost $M > |U|$ and all others cost one forces the union optimum to exceed the scenario-wise optimum or be infeasible. If $I(Q)$ is empty, the union is infeasible.

For a concrete family, put the path $s \to r \to t$ in both scenarios. Add $s \to a$ and $b \to t$ to the first and $a \to b$ to the second. Each scenario has only the path through r. The union also contains $s \to a \to b \to t$. Set $c(r) = 1$ and $c(a) = c(b) = M$. The optimum costs are one and $1 + M$, respectively. The gap follows from an incompatible provenance hypothesis, even though each edge is individually possible.

3 Computing a revalidation plan

A single scenario is a weighted vertex cut. Split each eligible vertex into an entrance and an exit joined by a capacity $c(v)$ edge. Give other edges capacity greater than the total eligible cost. Any affordable cut uses only the split edges and yields a revalidation plan by max-flow min-cut [1]. If even deleting all eligible vertices fails to disconnect the target, the problem is infeasible.

For multiple scenarios, start with $R = \varnothing$. If some graph still connects s to t, return one witness path P. Branch on adding each vertex of $I(P) \setminus R$. Every extension must choose at least one of these vertices, so taking the least-cost terminal solution is exact. If no path remains, the current set is a certificate. If the candidate branch set is empty, the current branch is infeasible. Caching by R is safe since the witness search depends only on the scenario family and R. With unit costs, at most ℓ eligible vertices per witness and a size cap k, search uses at most $1+\ell+\cdots+\ell^k$ reachability rounds. The parameter bound is useful when the smallest actionable witness has few eligible components, or when all witness paths are short.

The general decision problem is NP-complete, even with unit costs and a single three-edge path per scenario. Given a graph for vertex cover [2], make each graph vertex eligible. For every undirected edge $\{u, v\}$, choose its orientation using a common total order and create one scenario

with path $s \to u \to v \to t$. A revalidation set works precisely when it is a vertex cover. The reduction is polynomial, and a candidate solution is verified by reachability in each scenario.

4 Checking optimality without trusting the solver

A proposed set R of cost C gives an upper bound after reachability confirms that it blocks the target in every graph. For positive integer costs, proving optimality then amounts to ruling out every set of cost at most $C - 1$. Integer programming certificates support independent checks of such bounds [4]. Our certificate uses only operations visible in the recovery model.

At a node, some rebuilds S are already chosen and the remaining allowance is $b = C - 1 - \sum_{v \in S} c(v)$. A witness path must appear in one admitted scenario and avoid S. A branch node lists such a path and supplies one child for each eligible vertex on it. Every feasible completion must choose at least one of those vertices. Two kinds of leaf need only a local check. A negative allowance excludes the branch by cost, and a witness path with no eligible vertex excludes it by reachability.

A third kind of leaf uses nonnegative rational prices y_P on a finite list of residual witness paths. For each eligible vertex v, require

$$\sum_{P:\, v \in I(P)} y_P \le c(v).$$

Every completion T that hits those paths obeys

$$\sum_P y_P \le \sum_P y_P \, |I(P) \cap T| = \sum_{v \in T} \sum_{P:\, v \in I(P)} y_P \le \sum_{v \in T} c(v).$$

Therefore a total price strictly greater than b excludes the branch. The checker verifies the edges of each cited path against its named graph, so a path fabricated by mixing histories cannot justify a lower bound.

Theorem 2. *A feasible R of integer cost C and a valid lower-bound certificate for allowance $C - 1$ prove that R is optimal. Whenever R is optimal, a finite certificate exists using only branches, negative-allowance leaves and paths with no eligible vertex.*

Proof. Soundness follows by induction from the local tests and branch coverage. For completeness, branch on an available witness path as long as the allowance is nonnegative and its eligible set is nonempty. Each branch adds a vertex. If a branch within allowance had no witness path, it would be a feasible plan cheaper than R, contradicting optimality. The finite tree therefore closes at one of the two elementary leaf types. Rational prices are optional shortcuts and do not affect completeness. □

For three unit-cost vertices with residual path sets $\{a, b\}, \{a, c\}, \{b, c\}$, pricing every path at $1/2$ gives lower bound $3/2$. No unit-cost solution is possible. Any two vertices suffice. With two disjoint copies of this triangle, the true cost is four but root path prices cannot exceed three. Indeed, summing the six unit vertex capacities counts each path price twice. A branch can still complete the proof. The two examples distinguish rounding a lower bound from assuming the fractional problem solves every instance.

An independent checker validated the upper and lower certificates on 128 seeded graph pairs. It rejected five mutants, including a fabricated edge, an excessive price, a missing branch, a false cost prune and an infeasible proposed set. Pricing reduced the total number of nodes from 330 to 136. However, the compact JSON representation grew from 6,511 to 7,070 bytes across the fixture.

Prices saved bytes in 46 cases, increased them in 81, and tied in one. A one-node certificate can carry several paths, so node count alone is not a useful storage guarantee. An operator with a strict evidence budget can compare the sizes of two verified certificates before choosing which to store. The verifier checks the stated model. It cannot establish that the admitted dependency histories cover what happened in the incident.

4.1 Sharing proofs for the same rebuild set

For fixed histories, costs and starting allowance, a partial proof state is determined by the chosen set S. The allowance is the starting allowance minus $c(S)$, and the residual paths are those avoiding S. A verifier can therefore read one record per set S. At a branch with witness path P, the required child records are precisely $S \cup \{v\}$ for $v \in I(P)$. Each child adds a vertex, so the graph is acyclic. Induction in decreasing order of $|S|$ proves the same lower bound as the tree. For completeness, choose one valid subtree for each repeated state in a complete tree and retain only the records reachable from the empty set. Each shared record still has all the children its witness requires.

There is an exact separation for k disjoint triangles, with one scenario per edge and budget $2k-1$. Process the triangles in order. Within each triangle, branch first on $\{a, b\}$, then on the edge opposite the vertex just chosen. The tree has four occurrences of a two-vertex completion, while there are only three distinct pairs. Writing T_k for the number of tree nodes and D_k for the number of distinct states gives $T_0 = D_0 = 1$ and

$$T_k = 3 + 4T_{k-1} = 2 \cdot 4^k - 1, \qquad D_k = 3 + 3D_{k-1} = \frac{5 \cdot 3^k - 3}{2}.$$

The construction uses unit costs and a separate path s, u, v, t for each triangle edge $\{u, v\}$. Its optimum is $2k$ because any plan needs two vertices in each triangle.

In compact JSON, the five-triangle example used 2,047 tree nodes and 606 distinct states. A tree with a shared path dictionary used 9,798 bytes. Adding state sharing reduced it to 6,257 bytes, a reduction of 36.1%. The DAG was larger for one and two triangles. On the 128 seeded weighted graph pairs, it shared no states and used 5,069 bytes in total against 4,125 for the path-dictionary tree. The two encodings therefore favor different fixtures. The checker rejected seven negative controls, including an omitted child, a forged path, and use of the proof at a higher allowance. It also accepted a direct source-to-target witness with no eligible vertex.

5 Exact checks and use in incident response

The checker compares witness search with subset enumeration using ordered transitive closure. They agree for both the scenario-wise and union optima on all 192 seeded pairs. The union costs strictly more on 69 pairs, with a summed gap of 373 cost units and a maximum gap of 19. These are properties of the fixed synthetic design, not observed operational rates. The checker also verifies all 1,024 simple graphs on five labeled vertices against an independent vertex-cover oracle. In the explicit gap family it recovers costs one and 501 at $M = 500$. A separate check enumerates all 1,024 ordered pairs of forward four-vertex graphs without a direct source-to-target edge. Path enumeration tests the inclusion condition, while reachability tests feasible sets for all four choices of R. Both methods identify the same 252 pairs for which the feasible sets differ.

In practice, bind the plan to the set of dependency histories admitted by the evidence. If an operator learns a new possible history, run the witness search before reusing the old certificate. Retain a conservative scenario when a missing log could hide a path. The opportunity is to remove

impossible joint histories using trusted evidence, which can save full rebuilds. The danger is excluding a genuine history or treating a contaminated revalidation tool as independent. A prospective exercise should blind the solver to a known ground-truth log, inject both missing and mutually exclusive edges, and measure unnecessary rebuilds and remaining reachable taint. Use runtime isolation until the chosen reconstruction and the resumption authorization are checked.

Review status. Independent manuscript review is pending.

References

[1] L. R. Ford Jr. and D. R. Fulkerson. *Maximal Flow Through a Network*. Canadian Journal of Mathematics 8, 1956. `https://www.cs.yale.edu/homes/lans/readings/routing/ford-max_flow-1956.pdf`.

[2] Richard M. Karp. *Reducibility Among Combinatorial Problems*. Complexity of Computer Computations, 1972. `https://www.cs.umd.edu/~gasarch/BLOGPAPERS/Karp.pdf`.

[3] Naonori Kakimura and Ildikó Schlotter. *Parameterized Complexity of Submodular Minimization under Uncertainty*. 2024. `https://arxiv.org/abs/2404.07516`.

[4] Kevin K. H. Cheung, Ambros Gleixner, and Daniel E. Steffy. *Verifying Integer Programming Results*. 2017. `https://arxiv.org/abs/1611.08832`.